

Job Market Intelligence Pipeline

High-Level Design & System Architecture

Databricks | LangGraph | Databricks Vector Search | Python

March 2026

1. Executive Summary

This document describes the end-to-end architecture of the Job Market Intelligence Pipeline — a fully automated Databricks-native system that scrapes job postings from Google and Amazon, normalises and chunks them, embeds them into a vector index, and exposes a multi-agent LangGraph interface for natural language job market analysis.

The pipeline produces two primary outputs per query:

- A structured `final_answer` — a detailed analytical report based on retrieved job data.
- A `linkedin_post` — a polished social media post summarising the key insights.

Key Design Principles

- Idempotency — all pipeline stages use watermark-guarded MERGE operations; re-runs are safe.
- Incremental processing — each stage only processes data newer than its last watermark.
- Separation of concerns — scraping, cleaning, chunking, embedding, retrieval, and reasoning are fully decoupled.
- Observability — every stage writes status, row counts, and error notes to a watermark history audit table.

2. System Overview

The pipeline is composed of six sequential layers, each operating on Delta tables as the shared state store. Downstream layers are gated by upstream watermarks, ensuring they never process data before it is fully validated by the preceding stage.

Pipeline flow

Ingestion → Cleaning → Chunking → Embedding / Vector Index → Retrieval → Multi-Agent Reasoning

Layer	Scripts	Responsibility
1 — Ingestion	<code>scrape_google_careers.py</code> <code>scrape_amazon_careers.py</code>	Web scraping of Google and Amazon job listings; MERGE into raw_openings Delta table with dedup guard.
2 — Cleaning	<code>clean_loaded_data.py</code>	Normalisation of titles, locations, seniority, domain, and job family. Quarantine of invalid records.
3 — Chunking	<code>chunker.py</code>	Sentence-window chunking (window=2) with org-aware section detection. Idempotent MERGE into job_sentence_chunks.
4 — Embedding	<code>embedding.py</code> <code>vs_index.py</code> <code>vs_sync.py</code>	Databricks DELTA_SYNC vector index over job_sentence_chunks. Embedding via databricks-gte-large-en.
5 — Retrieval	<code>retriever.py</code>	Hybrid ANN + BM25 search with pre-filter (org, country, seniority, is_intern) and MMR re-ranking.
6 — Multi-agent	<code>build_agents.py</code> <code>08_run_queries.py</code>	LangGraph orchestrator routing to four specialist agents: Trend, Geo, Role, QA.

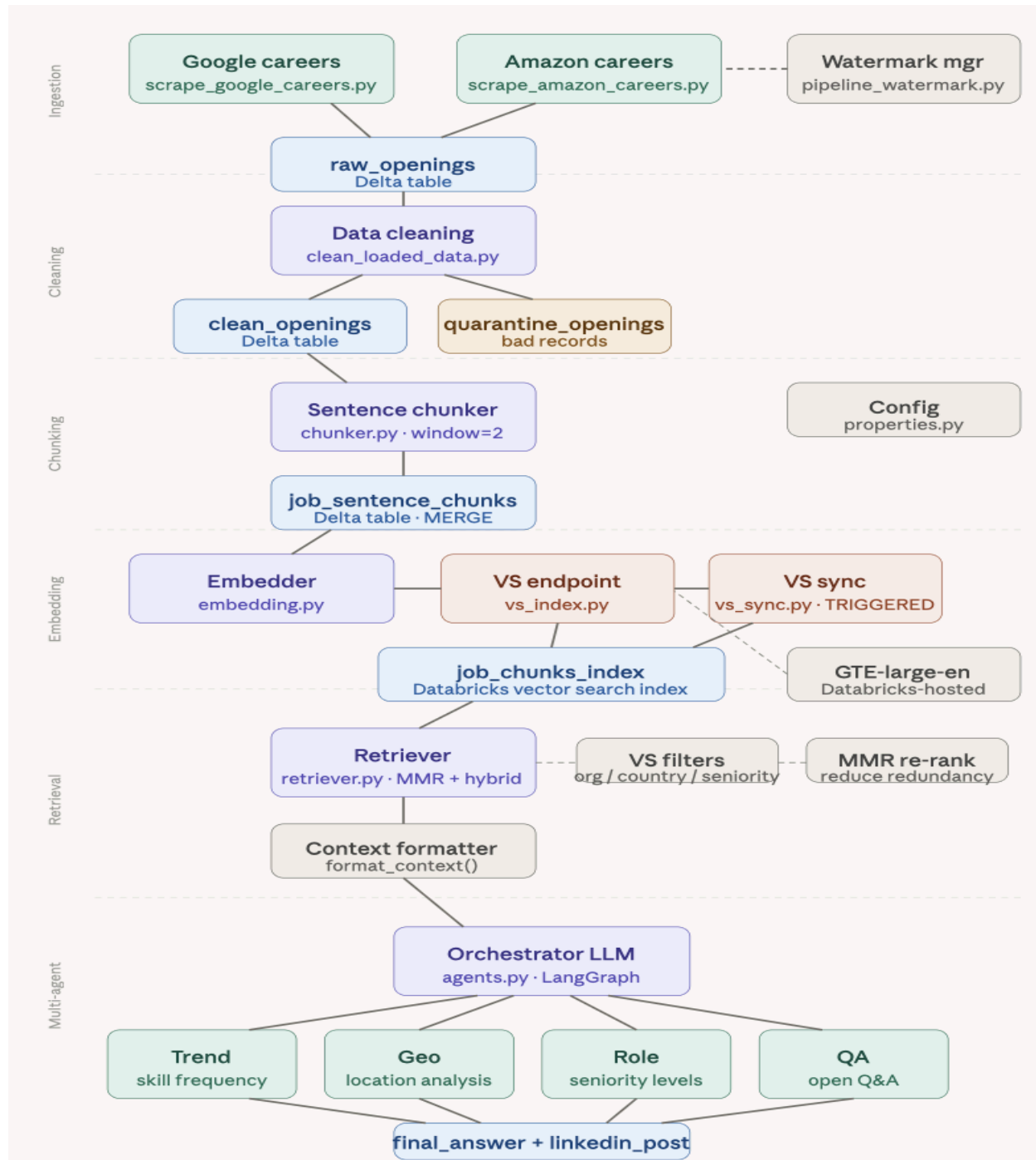


FIG HIGHLEVEL SYSTEM OVERVIEW

3. Component Design

3.1 Ingestion Layer

Two parallel scrapers collect job listings from publicly accessible career pages. Both scrapers register themselves with the WatermarkManager on startup and check raw_openings before scraping to avoid re-processing the same file_dt.

Google Careers Scraper — scrape_google_careers.py

- Iterates up to 10 pages per search term (data, ai, agentic, software).
- Extracts job title, organisation, location, position type, about, qualifications, and responsibilities using BeautifulSoup CSS selectors.
- Uses MERGE ON (job_id, organization) to deduplicate into raw_openings.
- Marks skipped / no_data / failed in the watermark table — never advances last_processed_dt.

Amazon Careers Scraper — scrape_amazon_careers.py

- Queries the amazon.jobs JSON search API across seven trend terms with pagination.
- Fetches full job description HTML per listing (rate-limited with exponential back-off).
- Merges on (id_icims, organization); all HTTP 4xx errors are swallowed gracefully.

Watermark Manager — pipeline_watermark.py

Shared across all pipeline stages. Tracks (org_key, stage) pairs in a Delta table. Provides mark_stage_success, mark_stage_failed, mark_skipped, mark_no_data, and status helpers. An append-only pipeline_watermark_history table provides a complete audit trail.

3.2 Cleaning Layer — clean_loaded_data.py

Reads raw_openings within the watermark window for a given org. Applies the following transformations:

- Title normalisation — maps variant job titles to canonical forms.
- Location parsing — extracts city and country from free-text job_location.
- Seniority detection — regex + LLM heuristics to classify I / II / III / Senior / Staff.
- Job family classification — Data Engineering, Software Engineering, Data Science, AI/GenAI, Data Center Operations.
- Domain / team extraction — Fintech, AWS, Advertising, etc. for Amazon.
- Intern flag — boolean is_intern derived from title and description keywords.

Records that fail minimum quality checks (missing title, empty description) are routed to quarantine_openings with a structured quarantine_reason.

3.3 Chunking Layer — chunker.py

Implements sentence-window chunking — a retrieval-optimised strategy where each chunk embeds only the focal sentence but stores a wider context window for LLM prompts.

Chunking Strategy

- window_size = 2: each focal sentence S[i] is stored with S[i-2] through S[i+2] as window_text.
- sentence = S[i] — the text that is embedded into the vector index.
- window_text = S[i-2] ... >>>S[i]<<< ... S[i+2] — passed to the LLM for context-rich generation.
- Section detection: Google and Amazon JDs are split by section headers (Minimum Qualifications, Key Job Responsibilities, About the Team, etc.) before chunking.
- Boilerplate removal: Amazon equal-opportunity and pay-range paragraphs are stripped before chunking.
- MERGE ON (doc_id, chunk_id) ensures idempotency — re-running chunker never produces duplicates.

3.4 Embedding & Vector Search — embedding.py / vs_index.py / vs_sync.py

Databricks Vector Search (DELTA_SYNC) automatically re-embeds job_sentence_chunks whenever a sync is triggered. No manual embedding loop is required.

- Index type: DELTA_SYNC with TRIGGERED pipeline — sync is explicit, not continuous.
- Embedding model: databricks-gte-large-en (Databricks-hosted, no external API key).
- Embedded column: sentence (focal sentence only — clean and dense).
- Metadata synced: chunk_id, doc_id, org_key, job_family, domain, seniority, is_intern, country, trend_dt, file_dt — all available as filter columns at query time.
- Sync polling: vs_sync.py polls detailed_state every 30 seconds up to 60 minutes, advancing the vectorize watermark only on ONLINE_NO_PENDING_UPDATE.

3.5 Retrieval Layer — retriever.py

Provides the retrieve() function consumed by the multi-agent layer. Implements a three-stage retrieval pipeline:

Stage 1 — Pre-filter (VS native filters)

Filters applied directly inside the vector search query for maximum performance:

- org_key: equality filter (google / amazon).
- job_family: normalised to canonical index values (Data Engineering, Software Engineering, etc.).
- country, section_type, trend_dt range, is_intern boolean.

Stage 2 — Candidate pool retrieval

Retrieves top_k × 4 candidates (default 60) using HYBRID query_type (dense ANN + BM25 lexical). Hybrid mode improves recall for rare technical terms (dbt, Iceberg, Medallion) that dense search alone misses.

Stage 3 — MMR re-ranking

Maximal Marginal Relevance re-ranks the candidate pool to balance relevance and diversity. Embeds all candidates + the query via the Databricks Foundation Model serving endpoint, then iteratively selects the next chunk that maximises $\text{lambda_mult} \times \text{relevance} - (1 - \text{lambda_mult}) \times \text{max_similarity_to_selected}$. Default $\text{lambda_mult} = 0.5$. This prevents all 15 returned chunks from coming from the same near-duplicate job posting.

3.6 Multi-Agent Layer — build_agents.py

A LangGraph StateGraph orchestrates four specialist tools. The orchestrator LLM decides which tools to invoke based on the user's natural language query — it may call one, several, or all four in sequence.

Agent	Tool name	Purpose
Orchestrator	<code>LangGraph graph</code>	Routes the query to the right combination of specialist tools. Synthesises tool outputs into <code>final_answer</code> and <code>linkedin_post</code> .
Trend agent	<code>trend_analysis</code>	Analyses skill frequency, technology mentions, and Python/AWS/SQL demand across all retrieved chunks.
Geo agent	<code>geo_analysis</code>	Identifies hiring locations, country distribution, and regional skill differences (US vs India, etc.).
Role agent	<code>role_analysis</code>	Compares seniority levels (Engineer I / II / III / Senior / Staff), responsibilities, and domain teams.
QA agent	<code>qa_tool</code>	Open-ended factual Q&A — answers specific questions not covered by the three specialist agents.

4. End-to-End Data Flow

The following table traces data from web scrape through to the final LinkedIn post.

Step	From	To	Mechanism
1	Google / Amazon career pages	<code>raw_openings</code> (Delta)	HTTP scrape → BeautifulSoup parse → MERGE ON (<code>job_id</code> , <code>org</code>)
2	<code>raw_openings</code>	<code>clean_openings</code>	Normalisation, seniority detection, domain classification, quality checks
3	<code>clean_openings</code>	<code>quarantine_openings</code>	Records failing minimum quality gates (missing title, empty description)
4	<code>clean_openings</code>	<code>job_sentence_chunks</code> (Delta)	Sentence-window chunking with section detection; MERGE ON (<code>doc_id</code> , <code>chunk_id</code>)

5	job_sentence_chunks	job_chunks_index (VS)	DELTA_SYNC triggered sync; databricks-gte-large-en embeds sentence column
6	User query (NL)	Retrieved chunks	Hybrid ANN+BM25 → pre-filter → MMR re-rank → format_context()
7	Retrieved chunks	Tool outputs (ToolMessage)	LangGraph orchestrator invokes trend / geo / role / qa agents
8	Tool outputs	final_answer + linkedin_post	Orchestrator LLM synthesises all tool results into two final outputs

5. Delta Table Schema Summary

Component	File(s)	Technology	Key detail
raw_openings	Scrapers	Delta + CDF	source of truth; one row per job posting
clean_openings	clean_loaded_data.py	Delta + CDF	normalised; all scrape orgs merged here
quarantine_openings	clean_loaded_data.py	Delta (no CDF)	bad records with quarantine_reason
job_sentence_chunks	chunker.py	Delta + CDF	one row per sentence chunk; MERGE idempotent
pipeline_watermarks	WatermarkManager	Delta + CDF	one row per (org_key, stage); UPSERT
pipeline_watermark_history	WatermarkManager	Delta (append)	complete audit log of every run attempt
job_chunks_index	embedding.py	Databricks VS	DELTA_SYNC; sentence column embedded
agent_query_results	08_run_queries.py	Delta (append)	saved query results for reporting / dashboards

6. Watermark & Idempotency Design

The watermark system is the backbone of the pipeline's safety guarantees. Every stage reads a lower-bound watermark (last successfully processed date) and an upper-bound watermark (the minimum of all upstream stages' watermarks) before deciding how much data to process.

Watermark Gate Logic

- Scraping: does not advance last_processed_dt. Only marks skipped / no_data / failed.
- Cleaning (per org): upper = today's file_dt. Lower = last normalize watermark for that org.
- Chunking (_pipeline): upper = MIN(all org normalize watermarks). Lower = chunk watermark.
- Vectorising (_pipeline): upper = chunk watermark. Lower = vectorize watermark.
- Consequence: adding a new org (Microsoft) automatically tightens the chunk upper bound until that org's normalize watermark catches up.
- Failed runs leave the watermark unchanged — the next run automatically retries the same window.

7. Technology Stack

Component	File(s)	Technology	Key detail
Compute	Databricks (serverless)	Notebooks / Jobs	All pipeline stages run as Databricks jobs
Storage	Delta Lake	Delta tables + CDF	ACID, time-travel, change data feed
Embedding model	Databricks Foundation Models	databricks-gte-large-en	No external API key; Databricks-hosted
Vector search	Databricks Vector Search	DELTA_SYNC / TRIGGERED	Auto-syncs from Delta; hybrid ANN+BM25
Agent framework	LangGraph	StateGraph + ToolNode	Multi-agent orchestration with message state
LLM client	databricks-langchain	ChatDatabricks	Unified LLM interface on Databricks
Web scraping	requests + BeautifulSoup	Python	Google and Amazon career pages
Tracing / MLOps	MLflow	langchain.autolog()	Token counts, latency, tool call traces
Configuration	properties.py	Python constants	Catalog, schema, index, endpoint names

8. Running the Pipeline

8.1 One-time setup

- Run `bootstrap_watermark.py` — creates all Delta tables and seeds watermark rows.
- Ensure the Databricks VS endpoint (`zachy_vs`) exists — `vs_index.py` creates it idempotently.

8.2 Daily pipeline run order

1. `scrape_google_careers.py` (parallel with step 2)
2. `scrape_amazon_careers.py`
3. `clean_loaded_data.py`
4. `chunker.py`
5. `embedding.py` (triggers VS sync; waits for completion)
6. `08_run_queries.py` (run queries; results saved to `agent_query_results`)

8.3 Example query usage

Install required packages before querying:

```
%pip install --upgrade databricks-langchain langchain langgraph
```

Run a natural language query:

```
from agents import run_queryresult = run_query("What are the top Python and  
Spark skills Amazon is hiring  
for?")print(result["final_answer"])print(result["linkedin_post"])
```

9. Acknowledgements

This pipeline was developed as part of the DataExpert bootcamp programme. Special thanks to Zach Willson, Umar Assis and the entire DataExpert team for their guidance, course content, and community support throughout this project.

End of Document